

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.16

GroupBy & Aggregation

Day 17 — *split, apply, combine*

90 min

Concepts + live code

30 min

Practice notebook

3

Words: split, apply, combine

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Remember this, from Day 15's practice?

What you wrote two days ago

```
for city in df["city"].unique():
    mask = df["city"] == city
    print(city, df.loc[mask, "python"].mean())

# four lines, one loop, manual printing
```

It worked. There is a much better way.

What you will write today

```
df.groupby("city")["python"].mean()

# one line, and the result is a
# Series you can sort, filter or plot
```

And it extends to two subjects, three statistics, or two grouping columns without getting any longer.

Three quick questions from Day 16 first

- 1 loc or iloc for the 3rd row?
- 2 Why can loc[2] fail after a filter?
- 3 Why does df[mask]["c"] = 5 do nothing?

→ iloc — you are counting.

→ Label 2 was removed; positions renumbered.

→ It writes to a copy. Use one .loc.

How today's two hours are structured

1

Split, apply, combine

The mental model behind every groupby

20 min

2

Aggregating

One function, many, and named results

25 min

3

Grouping by two things

MultIndex, unstack, pivot_table

20 min

4

transform & pitfalls

Broadcasting back, and missing keys

20 min

5

Mini build

A department report from raw rows

5 min

6

Practice notebook

You code, I walk around

30 min

Section 4 is marked red for one reason: groupby silently discards rows whose group key is missing.

By the end of today you will be able to

1

Group

Split a table by one or more columns and summarise each group

2

Aggregate

Apply several functions at once and name the results

3

Reshape

Move between long and wide with `unstack` and `pivot_table`

4

Compare

Put a group summary back beside every original row

Why this is the session that changes how you think

Almost every real question about data is a grouped question. Sales per region. Average delay per airline. Churn rate per plan. Once you see “per” in a question, you are looking at a `groupby` — and the answer is usually one line. Learning to hear that word is worth more than memorising the syntax.

1

Split, Apply, Combine

The mental model · 20 minutes

- The three steps
- What groupby returns
- size vs count

Split → apply → combine

```
df.groupby("city")["python"].mean()
```

1. SPLIT

	city	py
0	Pune	88
3	Pune	67
	city	py
1	Mumbai	91
5	Mumbai	58
	city	py
2	Delhi	45
4	Delhi	72

2. APPLY

mean of each
little table

```
Pune    77.5  
Mumbai  74.5  
Delhi   58.5
```

3. COMBINE

one row per group, the key becomes the index

```
city  
Delhi    58.5  
Mumbai   74.5  
Pune     77.5  
Name: python, dtype: float64
```

You never see step 1. pandas splits, applies and recombines in one operation — you only write the grouping column and the function. Note the result is sorted by GROUP KEY, not by value, and the key has become the index.

What groupby gives you — and size vs count

The intermediate object is lazy

```
gb = df.groupby("city")
gb
-> <DataFrameGroupBy object>

gb.ngroups    -> 3
gb.groups     -> {"Delhi": [2, 4], ...}
```

Nothing is computed until you ask for a statistic. Printing the groupby itself shows only an object.

⚠ size() and count() are different

```
# Amit's python mark is missing

gb.size()           Delhi 2    ROWS
gb["python"].count() Delhi 1    VALUES

# size counts rows, including empty ones
# count counts non-missing values
```

If the two disagree, you have missing data.

Two ways to count, with different sorting

```
df["city"].value_counts()    sorted by COUNT
df.groupby("city").size()    sorted by KEY
```

They give the same numbers in a different order. Use `value_counts` when you want to see the most common thing first; use `groupby().size()` when you are about to join the result to other grouped statistics and want a consistent order. Day 15's `value_counts` was a groupby all along.

2

Aggregating

One function, or many · 25 minutes

- Several columns
- agg with a list
- agg with a dict
- Named aggregation

From one number to a whole summary table

Several columns at once

```
df.groupby("city")[["python", "stats"]].mean()

# one row per city, one column per subject
```

Several functions on one column

```
df.groupby("city")["python"].agg(["mean", "max", "count"])
```

A different function per column

```
df.groupby("city").agg({
    "python": "mean",
    "stats": "max"})
```

Day 5's dictionary, telling pandas what to do with each column.

Any of mean, median, sum, min, max, std, count, size, first, last, nunique.

Named aggregation — the version worth defaulting to

```
df.groupby("city").agg(
    avg_python = ("python", "mean"),
    best_stats = ("stats", "max"),
    n           = ("name", "size"),
)
```

Each result gets the name YOU choose, so the output columns read like a report instead of mean, max, size. It also avoids the awkward two-level column headings that `agg(["mean", "max"])` produces on several columns at once.

The group key becomes the index — unless you say otherwise

Default: the key is the INDEX

```
df.groupby("city")["python"].mean()

city
Delhi      58.5
Mumbai     74.5
Pune       77.5

# "city" is the index, not a column
```

Good for `.loc["Pune"]` lookups and for plotting.

Two ways to get a flat table

```
df.groupby("city", as_index=False)\
    ["python"].mean()

# or, equivalently:
df.groupby("city")["python"]\
    .mean().reset_index()

# columns: ['city', 'python']
```

Use one of these when the result is going into another merge, or straight to a CSV.

Sorting: by key, not by value

```
# alphabetical by default: Delhi, Mumbai, Pune

df.groupby("city")["python"].mean()\
    .sort_values(ascending=False)    # by VALUE
```

groupby sorts by the group key so that repeated runs give a stable order. When you want the biggest group first, sort the RESULT — it is just a Series, so every Day 16 tool still applies. Passing `sort=False` keeps the order the keys first appeared in, and is slightly faster on large data.

3

Two Grouping Columns

MultIndex, unstack, pivot_table · 20 minutes

- Grouping by two
- unstack
- pivot_table

Grouping by two columns

Long form — a row per combination

```
df.groupby(["city", "grade"])["python"].mean()
```

city	grade	
Delhi	B	72.0
	C	45.0
Mumbai	A	91.0
	B	58.0
Pune	A	88.0
	B	67.0

The index now has TWO levels. The blanks mean “same as above”.

Wide form — unstack()

```
...unstack()
```

grade	A	B	C
city			
Delhi	NaN	72.0	45.0
Mumbai	91.0	58.0	NaN
Pune	88.0	67.0	NaN

unstack moves the LAST index level up into the columns. The NaNs are combinations that simply do not exist — no Delhi student got an A.

pivot_table does the same thing in one step

```
df.pivot_table(index="city", columns="grade",
               values="python", aggfunc="mean")
```

Identical result to groupby-then-unstack, written the way a spreadsheet user would think about it. Use whichever reads more clearly; pivot_table also takes fill_value=0 to replace those NaNs.

4

transform & Pitfalls

Back to the original shape · 20 minutes

- transform
- Missing group keys
- Common errors

transform — put the group answer beside every row

The problem agg cannot solve

“Which students scored above their OWN city's average?”

```
avg = df.groupby("city")["python"].mean()

# 3 rows. df has 6. They will not line up.
# agg SHRINKS the table - that is its job.
```

You need one value per ROW, not one per group.

transform keeps the shape

```
df["city_avg"] = (df.groupby("city")
                  ["python"]
                  .transform("mean"))
```

name	city	python	city_avg
Ravi	Pune	88	77.5
Neha	Pune	67	77.5
Sara	Mumbai	91	74.5

Six rows in, six rows out — the group's answer repeated for each of its members.

And now the question is one line

```
df[df["python"] > df["city_avg"]]["name"]
```

agg answers questions ABOUT groups. transform answers questions about rows that need to know something about their group. That is the whole distinction, and it is worth saying in those words.

The pitfall — rows with a missing group key disappear

! Silently, and without warning

```
# Ravi's city is missing

df.groupby("city")["python"].count()
Delhi    2
Mumbai   2
Pune     1    <- was 2

# total 5, but the table has 6 rows
```

No error. No warning. The row is simply not in any group.

Two defences

```
df.groupby("city", dropna=False)
...
NaN      1    <- now visible
```

```
# or check the arithmetic:
result.sum() == len(df)
```

Comparing the grouped total against the row count is the cheapest audit there is.

The third silent-missing-data trap of the week. Day 14: np.nansum of an all-missing column returns 0.0. Day 15: pandas skips nan in a mean without telling you. Today: a missing group key removes the row from the report entirely. The pattern is always the same — the number looks reasonable and nothing complains. Check your counts.

Debug corner — eight groupby bugs

You wrote	What happens	Fix
<code>df.groupby("city")</code>	prints an object, not a table	add a statistic: <code>.mean()</code>
rows with a missing key	silently dropped from the result	<code>dropna=False</code>
<code>size()</code> and <code>count()</code> mixed up	size counts rows, count counts values	pick the one you meant
assuming sorted by value	it is sorted by group KEY	<code>.sort_values()</code>
agg after picking one column	cannot use a dict of columns	put agg before the selection
transform expecting a summary	it returns the ORIGINAL shape	<code>.agg()</code>
<code>groupby(...)["a", "b"]</code>	KeyError — that is a tuple	<code>[["a", "b"]]</code>
<code>result["city"]</code> after groupby	city is the INDEX, not a column	<code>reset_index()</code>

Row 2 is the one that quietly produces a wrong report. All eight are in today's practice notebook.

Putting it together — a department report

```
import pandas as pd

staff = pd.read_csv("staff.csv")

# ---- 0. audit BEFORE grouping
print("rows:", len(staff))
print("missing dept:", staff["dept"].isna().sum())

# ---- 1. one summary row per department
report = staff.groupby("dept", dropna=False).agg(
    headcount = ("name", "size"),
    paid       = ("salary", "count"),      # non-missing
    avg_salary = ("salary", "mean"),
    top_salary = ("salary", "max"),
).round(0)

# ---- 2. the audit that catches dropped rows
assert report["headcount"].sum() == len(staff)

# ---- 3. sort by what matters, not by name
report = report.sort_values("avg_salary", ascending=False)
print(report)

# ---- 4. who beats their own department average?
staff["dept_avg"] = staff.groupby("dept")["salary"].transform("mean")
print(staff[staff["salary"] > staff["dept_avg"]]["name"].tolist())
```

**Audit first**

count the rows before you group

**dropna=False**

missing departments stay visible

**Named aggregation**

columns read like a report

**size AND count**

the gap reveals missing salaries

**assert**

the report must account for every row

**sort_values**

groupby sorts by key, not value

**transform**

per-row question, per-group answer

The assert is the part most people leave out. It turns a silent wrong answer into a loud failure.

Practice session — 30 minutes



Predict the output

Shapes, sorting, size vs count, transform

8 questions



Spot & fix the bug

Missing keys, index vs column, agg misuse

4 questions



Write the code

Group, aggregate, name, sort, pivot

5 tasks

Your notebook

[Day17_Practice_Questions.ipynb](#)

- Listen for the word “per”
- Check the grouped total = row count
- Raise a hand — I am walking around
- Solutions shared after class

After every groupby, check that the counts still add up to the number of rows you started with.

TODAY IN TWELVE LINES

Recap

- 1 groupby = split, apply, combine — you write one line.
- 2 Hear the word “per” in a question and reach for groupby.
- 3 The group key becomes the INDEX of the result.
- 4 Results are sorted by KEY, not by value. Sort them yourself.
- 5 size() counts rows; count() counts non-missing values.
- 6 If those two disagree, you have missing data.
- 7 agg(["mean","max"]) for many functions on one column.
- 8 agg({"a":"mean"}) for a different function per column.
- 9 Named aggregation gives your own column names — prefer it.
- 10 Two keys give a MultiIndex; unstack or pivot_table widens it.
- 11 agg SHRINKS the table; transform keeps the original shape.
- 12 ⚠ Rows with a missing group key vanish. Use dropna=False.

Homework

Rewrite Day 15's loop-over-cities as a single groupby · Build a report with four named aggregations and an assert that the counts add up · Use transform to find every row above its own group's average

Next class — Topic 1.17

Merge, join & concat: combining two tables on a shared key — and what happens to the rows that do not match